

Chapter 1: Frontend Layout & Navigation

Welcome to the CinéConnect tutorial! In this first chapter, we're going to explore how our movie app is structured visually. Think of it like building a house: before you can put furniture in or decorate, you need a strong frame and a way to move between rooms. That's exactly what "Frontend Layout & Navigation" is all about!

The Foundation of Our App: A Consistent Look and Feel

Imagine if every page in a website looked completely different, with menus appearing in new places, or the overall size changing all the time. It would be confusing, right? To prevent this, we create a consistent "shell" for our app. This shell provides a stable frame for all your content and a clear way to navigate.

Our main goal in this chapter is to understand how CinéConnect provides a consistent layout and an intuitive navigation system that adapts to different screen sizes.

Let's dive into the two main pieces that make this happen: **AppShell** and **SidebarNav**.

What is **AppShell**?

The **AppShell** (short for "Application Shell") is like the main frame or container of your entire app. When you open CinéConnect, most of what you see – the content area, the navigation – lives *inside* this **AppShell**.

It's designed to give your app a consistent look, feel, and structure, no matter which page you're visiting. It ensures that elements like the navigation menu are always in their expected place, and your main content has a dedicated area to display.

What is **SidebarNav**?

SidebarNav is our app's primary navigation system. It's the main menu that helps you jump between different sections of the app, like "Home", "Search", "Favorites", and your "Profile".

But here's where it gets clever: **SidebarNav** is designed to be **responsive**. This means it automatically changes its appearance to best suit the screen size you're using:

- **On Desktops (larger screens):** It appears as a slim bar on the left side, showing only icons. When you hover your mouse over it (or use your keyboard to focus on it), it gracefully expands to show the full labels for each menu item.
- **On Mobile Devices (smaller screens):** It transforms into a "drawer" menu. This means it hides the navigation initially, and you tap a menu icon (often called a "hamburger" icon) to slide it open from the left side, revealing the full text labels for all navigation items.

This adaptive design ensures a great user experience whether you're on a large monitor or a small phone.

How CinéConnect Uses the **AppShell** and **SidebarNav**

Let's see how these components are put together in our application's main layout.

At the very root of our frontend application, we have a component called `RootLayout`. This `RootLayout` decides *when* to use the `AppShell` to wrap content and when not to. For example, login and registration pages often don't need the full app shell because they are standalone forms.

Here's a simplified look at how `RootLayout` uses `AppShell`:

```
// apps/frontend/src/routes/-RootLayout.tsx
import { Outlet } from "@tanstack/react-router";
import { AppShell } from "@components/layout/AppShell"; // Our main app frame
import { useAuth } from "@hooks/useAuth"; // To check if user is logged in

// ... (logic to decide if AppShell is needed)

export function RootLayout() {
  const { isAuthenticated } = useAuth(); // Are we logged in?
  const pathname = useRouterState({ select: (s) => s.location.pathname }); // What page are we on?

  // Decide if the AppShell should be used for the current page
  if (shouldUseAppShell(pathname, isAuthenticated)) {
    return (
      <AppShell>
        { /* The content for the current page will appear here */ }
        <Outlet />
      </AppShell>
    );
  }

  // For pages like login/register, no AppShell is used
  return <Outlet />;
}
```

In this snippet:

- `RootLayout` is a special component that sits at the very top of our app's structure.
- `shouldUseAppShell` is a function that checks if the current page (`pathname`) should be wrapped by the `AppShell`. For instance, login or register pages usually appear without the `AppShell`.
- If `AppShell` is needed, we wrap `<Outlet />` with it. `<Outlet />` is a special placeholder provided by our routing library (TanStack Router) where the content of the current page will be rendered. This means `AppShell` provides the consistent frame around whatever page the user is currently viewing.

Now, let's look inside the `AppShell` itself.

```
// apps/frontend/src/components/layout/AppShell.tsx
import type { ReactNode } from "react";
import { SidebarNav } from "../SidebarNav"; // Our smart navigation!

export function AppShell({ children }: AppShellProps) {
  return (
    <div className="relative flex min-h-dvh bg-app-base text-ink">
```

```

    { /* ... (Skip link for accessibility, not central to layout concept) ... */}

    { /* This is where our navigation lives! */}
    <SidebarNav />

    { /* This div holds the main content of your application */}
    <div className="relative flex min-h-0 min-w-0 flex-1 flex-col">
      { /* ... (Optional full-bleed backdrop layer) ... */}

      <main id="main-content" tabIndex={-1} className="relative z-10 flex min-h-
0 min-w-0 max-w-full flex-1 flex-col overflow-y-auto overflow-x-hidden pt-14
scroll-mt-4 outline-none md:pt-0">
        {children} { /* This is where the page content from <Outlet /> goes */}
      </main>
    </div>
  </div>
);
}

```

As you can see, `AppShell` has a very simple but powerful job:

- It places the `SidebarNav` component on the left side.
- It creates a `<main>` area for the `children` (which is the actual content of the page, coming from `<Outlet />` in `RootLayout`).

This setup ensures that `SidebarNav` is always present alongside your main content when `AppShell` is active.

Under the Hood: How Layout and Navigation Work

Let's trace what happens when you visit a page in CinéConnect that uses the `AppShell`.

AppShell Flow

This diagram shows the flow:

1. The `User` asks the `Browser` to go to a page.
2. The `Browser` loads our top-level `RootLayout`.
3. `RootLayout` decides if the `AppShell` should be used for this page.
4. If yes, `RootLayout` tells `AppShell` to render, passing the actual page content as its `children`.
5. `AppShell` then places `SidebarNav` and a main content area.
6. `SidebarNav` intelligently checks the screen size to display either the desktop rail or the mobile drawer.
7. Finally, the `Browser` shows the complete, well-structured page to the `User`.

SidebarNav: The Smart Navigator

The `SidebarNav` component is where the magic of responsive navigation happens. It uses a CSS breakpoint (specifically `md`, which usually means "medium-sized screens and larger") to decide which version of the navigation to display.

Let's look at the structure of `SidebarNav` and how it handles different screen sizes.

Desktop Navigation (Large Screens: **md** and up)

On larger screens (like desktops or tablets held horizontally), **SideNavBar** shows a slim, icon-only rail on the left. This rail expands when you hover your mouse over it or use keyboard navigation (focus).

```
// apps/frontend/src/components/layout/SideNavBar.tsx (Simplified)
// ... (imports) ...

export function SideNavBar() {
  // ... (state and data fetching) ...

  return (
    <>
      {/* Hidden on desktop: MobileNavDrawer is rendered here but hidden using CSS */}
      <div className="fixed inset-x-0 top-0 z-40 md:hidden">
        <MobileNavDrawer />
      </div>

      {/* Desktop sidebar: Visible on md screens and up */}
      <aside
        className={cn(
          "group/sidebar hidden h-dvh w-14 shrink-0 flex-col overflow-x-hidden
border-r bg-[var(--glass-bg)] backdrop-blur-[var(--glass-blur)] md:flex",
          "motion-safe:transition-[width] motion-safe:duration-300 motion-
safe:ease-out motion-reduce:transition-none",
          "md:hover:w-56 md:focus-within:w-56" // This makes it expand!
        )}
        aria-label="Navigation principale"
      >
        {/* ... (Logo and main navigation items) ... */}
        <nav className="flex flex-col gap-0.5 p-2">
          {MAIN_NAV.map((item) => (
            <NavItem key={item.to} item={item} pathname={pathname} />
          ))}
        </nav>
        {/* ... (Settings, Profile, Logout) ... */}
      </aside>
    </>
  );
}
```

In this part of **SideNavBar**:

- The **aside** element is the container for our desktop sidebar.
- The **hidden md:flex** classes mean: "hide this by default, but display it as a flex container on **md** screens and larger."
- The key to its expansion is **md:hover:w-56 md:focus-within:w-56**. This Tailwind CSS class tells the sidebar to change its width from a slim **w-14** (14 units wide) to a wider **w-56** (56 units wide) when a

user hovers over it or tabs into an element inside it. The `transition-[width]` class makes this change smooth and animated.

- Each `NavItem` inside the `nav` element then adapts its label visibility based on whether the `aside` is expanded.

Mobile Navigation (Small Screens: Below `md`)

On smaller screens (like phones), the desktop sidebar is hidden. Instead, a simple top bar appears with a menu button. Tapping this button opens a full-text "drawer" menu from the left.

```
// apps/frontend/src/components/layout/SidebarNav.tsx (Simplified -
MobileNavDrawer)
// ... (imports) ...

function MobileNavDrawer() {
  const [open, setOpen] = useState(false); // Manages if the drawer is open or
  closed
  // ... (profile and notification data) ...

  return (
    <div className="md:hidden"> {/* This whole component is hidden on md screens
    and up */}
      <Dialog.Root open={open} onOpenChange={setOpen}>
        <header className="flex h-14 min-h-14 shrink-0 items-center gap-2 border-b
        bg-[var(--glass-bg)]/95 px-3 backdrop-blur">
          <Dialog.Trigger asChild>
            <button type="button" aria-label="Ouvrir le menu de navigation">
              <Menu className="h-6 w-6 shrink-0" aria-hidden /> {/* The
              "hamburger" icon */}
            </button>
          </Dialog.Trigger>
          {/* ... (App title link) ... */}
        </header>

        <Dialog.Portal>
          <Dialog.Overlay /> {/* Darkens the background when menu is open */}
          <Dialog.Content className="!fixed !left-0 !top-0 z-[201] flex h-dvh max-
          h-dvh w-[min(20rem,calc(100vw-1rem))] flex-col !translate-x-0 !translate-y-0
          outline-none border-r bg-[var(--glass-bg)] shadow-2xl backdrop-blur">
            {/* ... (Drawer header with close button) ... */}

            <nav className="flex min-h-0 flex-1 flex-col gap-0.5 overflow-y-auto
            p-2">
              {MAIN_NAV.map((item) => (
                <NavItem
                  key={item.to}
                  item={item}
                  pathname={pathname}
                  variant="drawer" // Tells NavItem to always show the label
                  onMenuClose={() => setOpen(false)}
                />
              ))}
            </nav>
          </Dialog.Content>
        </Dialog.Portal>
      </Dialog.Root>
    </div>
  );
}
```

```

        </nav>
        { /* ... (Settings, Profile, Logout in drawer) ... */ }
      </Dialog.Content>
    </Dialog.Portal>
  </Dialog.Root>
</div>
);
}

```

Key points about **MobileNavDrawer**:

- The **md:hidden** class ensures this component *only* appears on screens smaller than **md**.
- It uses a **Dialog** component (from Radix UI) to create the drawer effect.
- The **Dialog.Trigger** is our "hamburger" menu icon (**<Menu />**). Tapping it sets **open** to **true**.
- When **open** is true, **Dialog.Content** slides in from the left, showing the navigation links.
- Notice the **variant="drawer"** prop passed to **NavItem**. This tells **NavItem** to always display the full text label, which is ideal for a mobile menu.

NavItem: The Individual Link

Both the desktop sidebar and the mobile drawer use the same **NavItem** component to render each link (Home, Search, etc.). The **variant** prop ("**rail**" for desktop, "**drawer**" for mobile) tells **NavItem** how to display itself.

```

// apps/frontend/src/components/layout/SidebarNav.tsx (Simplified NavItem)
import { Link } from "@tanstack/react-router"; // For navigation
import type { LucideIcon } from "lucide-react"; // For icons

export function NavItem({ item, pathname, variant = "rail", onMenuClose }: {
  item: NavItemConfig; // Contains 'to', 'label', 'icon'
  pathname: string;
  variant?: "rail" | "drawer"; // How it should look
  onMenuClose?: () => void; // For mobile, to close the menu
}) {
  const Icon = item.icon; // The icon for this navigation item
  const isDrawer = variant === "drawer";

  return (
    <Link
      to={item.to}
      title={item.label}
      aria-label={item.label}
      onClick={() => onMenuClose?.()} // Close drawer on mobile
      className={cn(
        isDrawer ? "flex items-center gap-3 px-3 py-2.5" : "justify-center px-1.5
md:group-hover/sidebar:justify-start md:group-hover/sidebar:gap-3 md:group-
hover/sidebar:pl-3", // Styling logic
        // ... (active state styling) ...
      )}
    >

```

```

    <span className="flex h-9 w-9 shrink-0 items-center justify-center">
      <Icon className="h-5 w-5 shrink-0" aria-hidden /> { /* The icon */}
    </span>
    <span
      className={cn(
        "min-w-0 truncate text-sm font-medium",
        isDrawer
          ? "flex-1 text-left opacity-100" // Always visible in drawer
          : "max-w-0 overflow-hidden opacity-0 md:group-hover/sidebar:max-w-[11rem] md:group-hover/sidebar:opacity-100" // Hidden then shown on desktop hover
      )}
    >
      {item.label} { /* The label text */}
    </span>
  </Link>
);
}
```

This `NavItem` smartly handles:

- **Icons:** Always visible.
- **Labels:** On desktop (`variant="rail"`), they are hidden by default (`opacity-0`, `max-w-0`) and only shown when the parent `aside` expands (`md:group-hover/sidebar:opacity-100`). On mobile (`variant="drawer"`), labels are always visible (`opacity-100`, `flex-1`).
- **Active State:** The component also applies special styling (like a red background and text) if the user is currently on that page.

Summary of Layout and Navigation Decisions

Here’s a quick overview of how CinéConnect adapts its layout:

Feature	Desktop (md+)	Mobile (Below md)
AppShell	Always wraps content	Always wraps content
Navigation Type	Fixed Left Sidebar Rail	Top Bar with Left Drawer Menu
Sidebar Display	Icon-only, expands on hover/focus to show labels	Hidden by default, slides open from left on menu tap
Labels	Hidden by default, shown on hover/focus	Always visible
Main Content	Appears to the right of the sidebar	Below the top bar, drawer overlays content when open

Conclusion

In this chapter, we've explored the foundational concepts of CinéConnect's frontend layout and navigation. We learned that the `AppShell` provides a consistent main frame for our application, ensuring a uniform visual structure across pages. We also saw how `SidebarNav` acts as our smart navigation system, cleverly adapting

its display for desktop (an expanding icon-only rail) and mobile devices (a left-hand drawer menu with full text labels). This adaptive design guarantees a smooth and intuitive user experience on any device.

Now that we understand how our app is structured visually, in the next chapter, we'll dive into how users can log in, register, and manage their sessions with [User Authentication](#).